UNIVERSITY MALAYSIA TERENGGANU
FACULTY OF COMPUTER SCIENCE & MATHEMATICS

SOFTWARE ARCHITECTURE
CSE3433

FINAL REPORT

GROUP NO. 11

SERVICE-ORIENTED SMART CAMPUS SUPPORT SYSTEM

Prepared by:
1. SARAH AMIRA HERI (S75812)
2. ELSA FADLIN BINTI MERSING (S75412)
3. SITI AISYAH ARWIAH BINTI YUNDI (S74256)

Prepared for:

DR. MOHAMAD NOR HASAN

[BACHELOR OF COMPUTER SCIENCE (SOFTWARE ENGINEERING) WITH HONOURS]
SEMESTER 2 2025/2026

Table Of Content

1.0 Introduction

The Smart Campus Support System is a centralized digital platform designed to improve and streamline campus services for students and staff. The system is developed using Service-Oriented Architecture (SOA), where different services such as user management, facility booking, complaint handling, and notifications function as independent modules but are connected through APIs.

This system integrates multiple campus services into a single platform, allowing users to access various functions through one interface instead of using separate systems. Each service focuses on a specific task while still being able to communicate with other services when needed. This modular design makes the system more flexible, scalable, and easier to maintain.

The system supports different types of users, mainly students and administrative staff, with different access levels. Students can register, manage their profiles, book facilities, submit complaints, and receive notifications. Meanwhile, administrative staff can manage user data, monitor complaints, and send announcements more efficiently.

In addition, the facility booking module allows users to check availability and make reservations in real time, which helps reduce scheduling conflicts. The complaint management feature also enables users to report issues and track their status, ensuring faster response and better transparency.

Overall, the Smart Campus Support System aims to replace manual and disconnected processes with a unified, efficient, and user-friendly solution that improves productivity and enhances the overall campus experience.

2.0 Problem Description

The old campus service management is very inconvenience as it largely managed through manual processes and separate platforms for each service. They all rely on physical paperwork, registration forms, and payment methods. This traditional approach resulted in inefficiencies, long waiting time, and limited accessibility. Therefore, the overall management is not suitable anymore in this modern world.

There are several problems with the old campus service management which is the lack of centralized system. This is because it uses multiple separate system that are not connected to each other. For example, the student and staff must use different platforms for different services. This leads to data inconsistency, duplication of information, and difficulty in accessing accurate data.

Other than that, the administrative services are often slow due to manual processes as it relies on physical interactions. Tasks such as registration, fee payment, and handling inquiries require students to queue at service counters, resulting in long waiting times and delays. This reduces productivity and creates inconvenience for both students and administrative staff.

Lastly, facility management in many campuses is inefficient due to the absence of a proper digital system. Booking rooms, reserving labs, or reporting maintenance issues are often done manually or through informal methods. This leads to scheduling conflicts and delayed maintenance responses, ultimately affecting the overall campus experience.

3.0 System Requirements

3.1 Functional Requirements

The system will include the following main functionalities:

1. User Management

- Students can register and log in
- Profile management

2. Facility Booking

- View available facilities (e.g., rooms, labs, halls)
- Book and cancel reservations

3. Complaint Management

- Submit complaints or issues
- Track complaint status

4. Notification System

- Receive announcements from administration
- Get alerts for booking status or complaint updates

5. Service Integration

- All services are connected through SOA

Each module works independently but communicates with others

3.2 Non-functional Requirements

Non-functional requirement define how the system should operate

i. Performance
   - The system should load pages quickly.
   - Communication between services is fast using lightweight APIs.
   - Latency between modules is minimized.

ii. Reliability
   - Each service operates independently; this ensures one service does not affect the overall system functionality.

iii. Scalability
   - Booking service can scale more during peak times.
   - Complaint service can scale independently if many users submit issues.

iv. Interoperability
   - Standardized communication protocols such as RESTful APIs enable seamless interaction between services.

v. Maintainability
   - System can be improved through a loosely coupled architecture.

4.0 Architecture Design

Service-Oriented Architecture (SOA) is a client/server design approach in which an application consists of software services and software service consumers. Instead of building a single, tightly integrated system, SOA divides functionalities into smaller services, each responsible for a specific task such as data processing, user management, or payment handling. These services can be accessed and combined to support different applications, making the system more flexible and easier to maintain.

4.1 Presentation Layer

The Presentation Layer acts as the direct boundary between the student and the application logic. This part is built using a combination of JSP, HTML5, CSS, and client-side JavaScript. It is responsible for the interface rendering and data collection. In the system, login.jsp, signup.jsp, and dashboard.jsp does not communicate with database directly but instead, they serve as layout templates. This feature dynamic placeholders that wait for data attributes pushed down by the controllers.

4.2 Application Layer

The Application Layer acts as the core of the entire system and is handled by Java Servlet files. When a student submits information from the screen, this layer catches it and checks it against the system's rules to make sure everything makes sense. For example, it is the controller that verifies if a student's booking end-time is accidentally set before the start-time, immediately stopping the mistake and sending back an error message. This layer also manages behind-the-scenes security, like scrambling user passwords into safe codes using SHA-256 encryption and keeping track of who is currently logged in so they do not get kicked out while navigating the site.

4.3 The Data Access Layer

The Data Access Layer is the storage manager of application, consisting of DAO files and the DBConnection utility. This is the only part of the program allowed to talk directly to MySQL databases. To keep the system organized, this layer splits the data into two completely separate storage rooms. Module A handles the complaints and facility bookings, while Module B securely stores student account profiles. When the controller layer needs info, a DAO will go

into the database, run a quick SQL query to find the requested rows, package that raw data into clean Java objects, and hand them back up to the system.

5.0 Architecture Diagrams

5.1 System Context Diagram

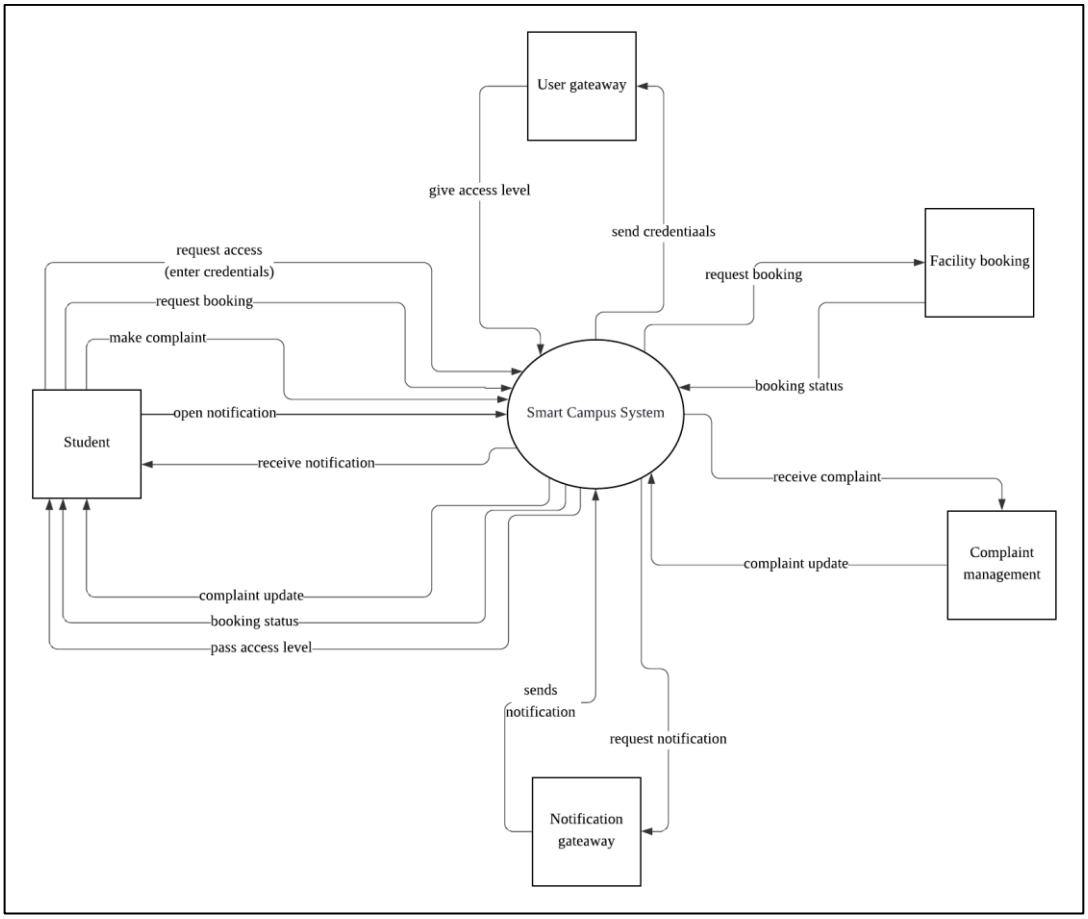

Figure 5.1.1 System Context Diagram

Figure 1 shows the System Context Diagram of the system. Student is the main user of the system. They'll use the system by requesting modules like access by entering their credentials, booking and make complaint or open their notification. This action will send their request to the system and the system will forward their request to the responsible 3rd party services. For example: when student want to book a facility, they'll request the booking module in the system. This request will be forwarded by the main system to an external faculty booking services. This services then will send back booking status to the main system and the system will forward booking status to the student booking.

5.2 Component/Service Architecture Diagram

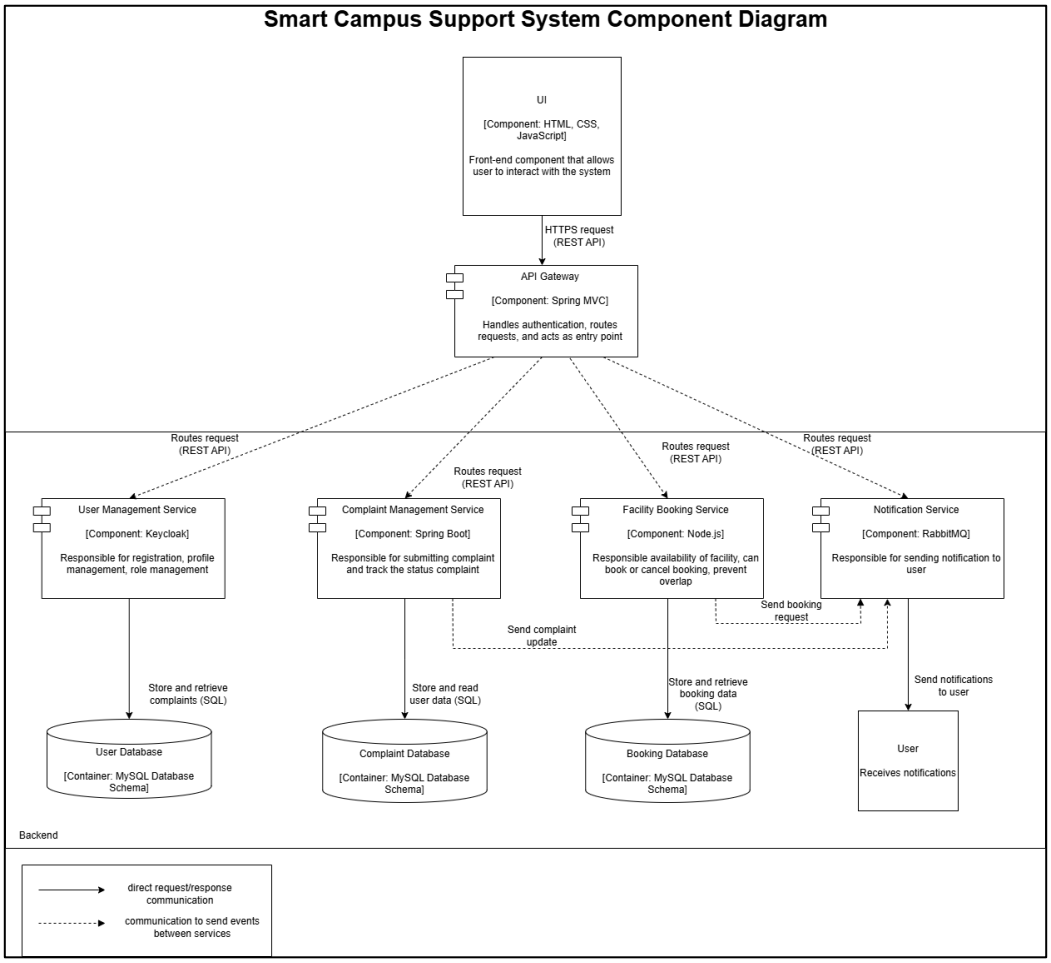

Figure 5.2.1 Component/Service Diagram

The system is designed using Service-Oriented Architecture, where each component is loosely coupled. The User Management Service handles authentication and user data, while the Facility Booking Service manages reservations. The Complaint Management Service handles issue reporting, and the Notification Service delivers system alerts. The API Gateway handles all incoming requests, which are routed to independent services such as User, Booking, and Complaint services via REST APIs. Each service maintains its own database. Event-driven communication allows services to send updates to the Notification Service, which delivers messages asynchronously to users.

5.3 Interaction Diagram

5.3.1 Booking Module Interaction Diagram

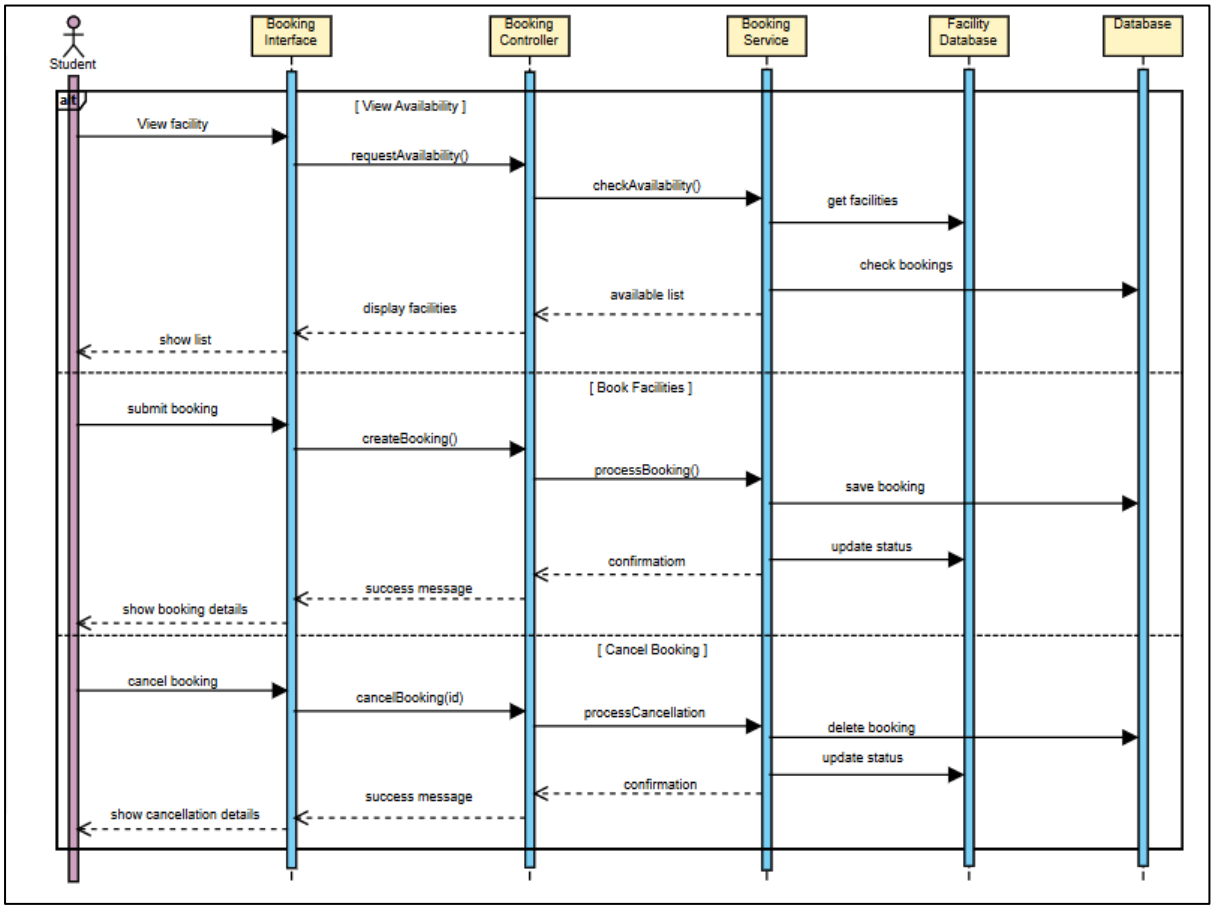

Figure 5.3.1.1 Booking Module Interaction Diagram

This interaction diagram shows how the Facility Booking system manages viewing facilities, booking, and cancellation using a simple flow. The student interacts with the booking interface, which sends requests to the controller and service layer for processing. The system checks availability, saves booking details, or cancels reservations while updating the facility and booking databases accordingly. The results are then returned to the student as confirmation or updated information. This diagram keeps the structure simple while still fulfilling the required system components.

5.3.2 Complaint Module Interaction Diagram

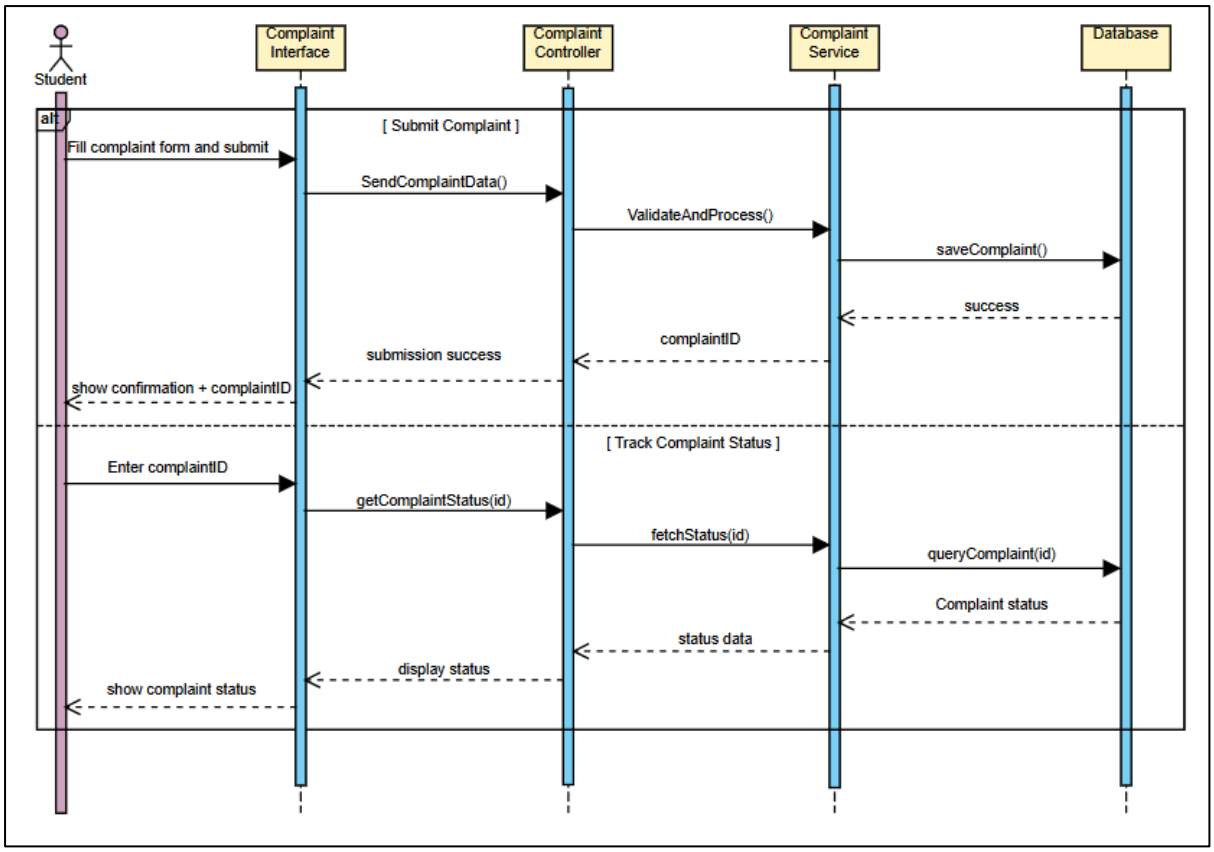

Figure 5.3.2.1 Complaint Module Interaction Diagram

The interaction diagram shows how the Complaint Management system handles two main functions which are submitting a complaint and tracking its status. When a student submits a complaint, the information is sent through the interface to the controller and service layer, where it is processed and stored in the database, and a confirmation along with a complaint ID is returned to the student. For tracking, the student enters the complaint ID, and the system retrieves the complaint details from the database through the same layers before displaying the current status. This diagram highlights how the system components work together to manage both processes efficiently in a single flow.

5.3.3 Notification Module Interaction Diagram

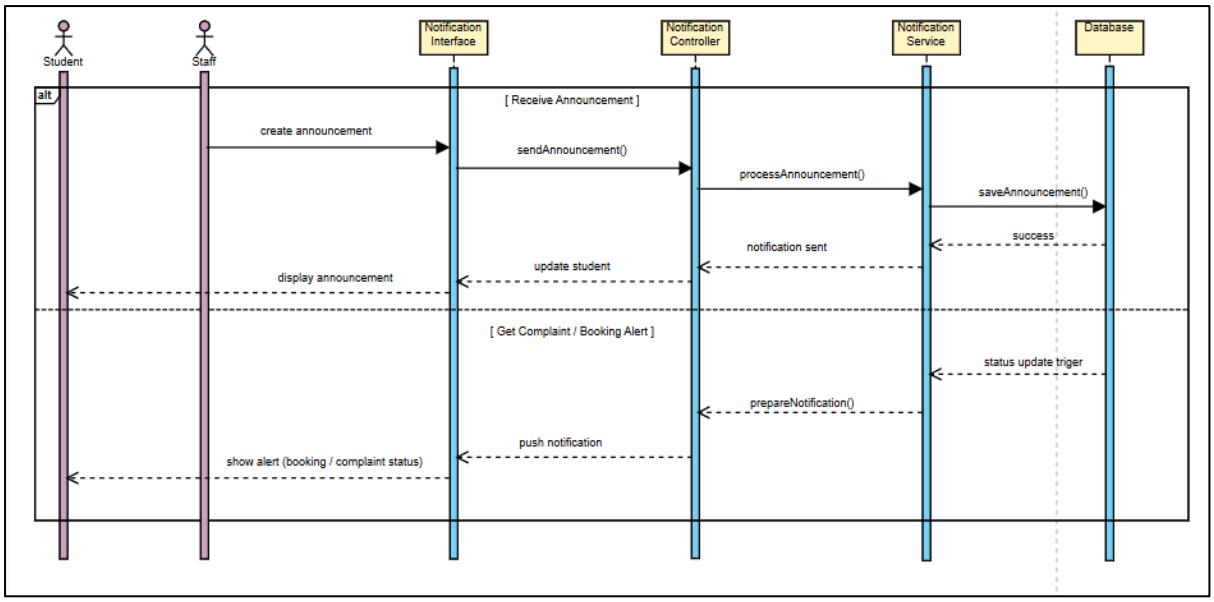

Figure 5.3.3.1 Notification Module Interaction Diagram

This interaction diagram shows how the Notification System manages both announcements and alert notifications. For announcements, the staff creates a message through the interface, which is processed, stored in the database, and then delivered to student. For alerts, the system automatically detects updates such as booking status or complaint progress from the database and sends notifications to student through the controller and interface. Overall, the diagram illustrates how the system ensures student stay informed by handling both manual announcements and automatic alerts in a single flow.

5.4 Deployment Diagram

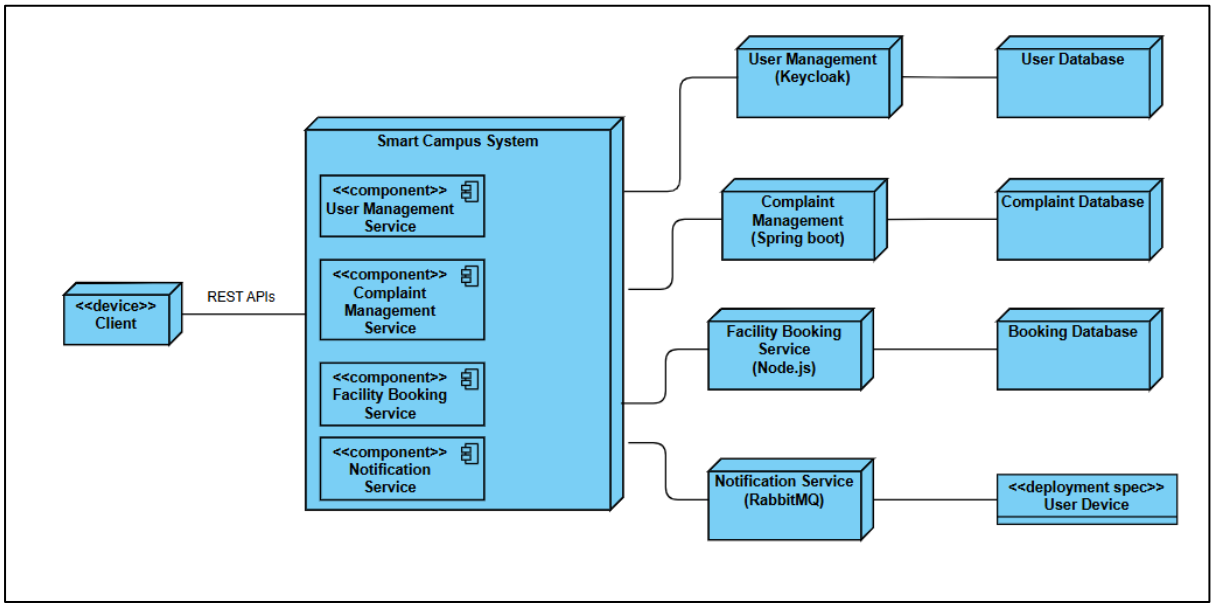

Figure 5.4.1. Deployment Diagram

Figure 6 shows the Deployment Diagram of the system. Client will access the system via REST APIs. The system is made up of 4 components and each of the components in the system will call on the 3rd party services to do the components function. For example: the notification service in the system will call out on RabbitMQ to send notification to the user.

6.0 Technology Stack

Architecture Layer	Technology Used	Purpose
Presentation Layer	JSP, HTML, CSS, JavaScript	Dynamic web pages (dashboard.jsp, login.jsp, signup.jsp) serving as the User Interface for user.
Application Layer	Java Servlets	Processes business logic and user requests.
Security Component	SHA-256 Hashing	Protects user passwords.
Session Management	HTTP Session	Maintains user login sessions.
Database Connectivity	JDBC	Connects the application to the database.
Data Layer	MySQL Server	Stores and manages application data.
Database Driver	MySQL Connector	Active driver dependency providing the standard execution link between Java application runtime and the MySQL instances.
Third-Party Service	UI Avatars API	Generates user profile avatars automatically.

7.0 Prototype Implementation

The Smart Campus Support System prototype was developed to show the core module of Smart Campus System. This prototype includes 3 modules: User Management Module, Booking Module, Complaint Module, and Notification Module.

7.1 User Management Module

User Management Module is responsible in handling user. The features inside user module include user registration, user login and dashboard display.

7.2 Booking Module

Booking Module is responsible for users to check any facility availability, booking them and track their booking request. Features inside booking module: Facility availability check, facility booking and booking request tracking.

7.3 Complaint Module

Complaint Module is responsible for users to submit their complaints and track if their complaints have been responded. Features inside complaint module: Complaint submission form and basic status tracking (pending or complete).

7.4 Notification Module

Notification Module is responsible in notifying the users, the system will feel more responsive with this module. Features inside Notification module: System alerts notification

8.0 Architecture Evaluation

The Service-Oriented Smart Campus Support System architecture was evaluated based on several quality attributes.

8.1 Performance

The system satisfies high performance and low-latency requirements by utilizing lightweight Java Servlets and direct database connectivity via JDBC and the MySQL Connector. This structured data pipeline minimizes processing overhead between the presentation layer and the database, enabling the complaint management web pages to load quickly. This ensures that student complaint submissions, status updates, and tracking requests are processed and saved with minimal delay under normal operating conditions.

8.2 Reliability

The architecture ensures a reliable environment for handling campus issues through isolated execution boundaries and strict input validation rules. By decoupling the Complaint Management module from other campus services, localized failures or unexpected downtimes in external components will not cause the complaint features to crash. Reliability is further reinforced at the application boundary by backend logic that validates incoming user data, ensuring that only users with a valid and registered account can successfully commit transactions to the database, thereby eliminating invalid data corruption or anonymous system abuse

8.3 Scalability

The service-oriented design allows individual service modules, such as the implemented Complaint Management Service, to scale independently during peak usage periods. By separating campus features into distinct, decoupled components, system resources can be expanded or focused specifically where data demand spikes such as high-volume complaint submission intervals without causing resource exhaustion or performance degradation across other campus operations.

8.4 Interoperability

Standardized communication protocols enable seamless interaction and data interchange across the system pipeline. The presentation layers written in JSP, HTML, CSS, and JavaScript

transfer parameters smoothly to backend Java Servlets and the Complaint Management database using traditional form data and standard HTTP request payloads. This setup ensures that native web tech stacks work reliably alongside external tools like the UI Avatars API, while web-tunneling tools like ngrok allow these service endpoints to securely bridge and interoperate across local and distributed network boundaries during integration testing.

8.5 Maintainability

The explicit separation of concerns achieved through a service-oriented approach significantly improves the overall maintainability of the platform. Because the core business logic is isolated within the Complaint Management Service, modifications, bug fixes, or future updates to the complaint submission workflows can be implemented cleanly. Developers can adjust backend Java Servlets or update individual logic parameters without risk of breaking or forcing modifications onto adjacent integrated services.

Overall, the architecture successfully meets the functional and non-functional requirements defined for the Service-Oriented Smart Campus Support System.

9.0 Team Contributions

Name	Matric No	Role	Task
Elsa Fadlin Binti Mersing	S75412	Frontend Developer & Tester	• Design and develop the user interface • Ensure system is user-friendly and responsive • Conduct system testing and debugging • Report and fix errors to improve system quality
Sarah Amira Heri	S75812	Project Manager & System Analyst	• Plan and manage the overall project timeline • Gather and analyze system requirements • Define system architecture (SOA design) • Ensure the project meets objectives
Siti Aisyah Arwiah Binti Yundi	S75256	Backend Developer & Database Designer	• Develop core system functionalities (services in SOA) • Handle server-side logic and integration between services

			• Design and manage the database structure • Ensure data consistency and security

10.0 Conclusion and Reflection

The development of the Service-Oriented Smart Campus Support System successfully replaces legacy, disconnected manual processes with a centralized digital platform. By adopting a Service-Oriented Architecture (SOA), the system successfully splits core functionalities into autonomous modules User Management, Facility Booking, Complaint Management, and Notification Services that interact smoothly via REST APIs and an API Gateway. As proven by our architecture evaluation, this loosely coupled structure achieves high reliability, independent scalability, and easy maintainability. Overall, the successful implementation of our prototype validates that transitioning to the service-oriented ecosystem is an efficient and viable approach to modernizing campus services.

Hands-on experience with this project bridged the gap between architectural theory and practical implementation. Designing the various architectural diagrams such as the System Context, Component, Interaction, and Deployment diagrams deepened our understanding of how enterprise-level systems are structured. We gained valuable technical insights into decoupling services, managing independent databases, and navigating interoperability across a diverse tech stack. Ultimately, evaluating the system against strict quality attributes taught us that software architecture is about balancing trade-offs to plan for system longevity, providing our team with a critical engineering mindset for our future careers.